Métodos Basados en

Árboles (CARTs)

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap

Recap

- Queremos predecir y en funcion de observables $(\mathbf{x}_i)$

$$y = f(\mathbf{x}_i) + u \tag{1}$$

- donde la estimación de f implica la que minimize el riesgo empírico (prediga mejor fuera de muestra):

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{2}$$

Recap

$f(x_i, \beta)$

- Por ejemplo en regresión $f(x_i) = \mathbf{x}_i\beta$ y minimizamos

$$\hat{\beta} = \underset{\tilde{\beta}}{\text{argmin}} \left\{ \sum_{i=1}^{n} (y_i - \mathbf{x}_i\beta)^2 \right\} \tag{3}$$

SRR
RSS

- Por ejemplo en clasificación logistica $p(x_i) = \frac{1}{1+e^{-\mathbf{x}_i\beta}}$

$$\hat{\beta}^{MLE} = \underset{\beta}{argmin} \left\{ - \left[\sum_{i=1}^{n} log \left(\frac{p_i}{1-p_i} \right)^{y_i} + \sum_{i=1}^{n} log(1-p_i) \right] \right\} \tag{4}$$

→ Grad. → Newton → H
↳ Des.

Agenda

1 Recap

2 Árboles
- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

3 Bagging y Random Forests
- Bagging
- Random Forests

4 Recap

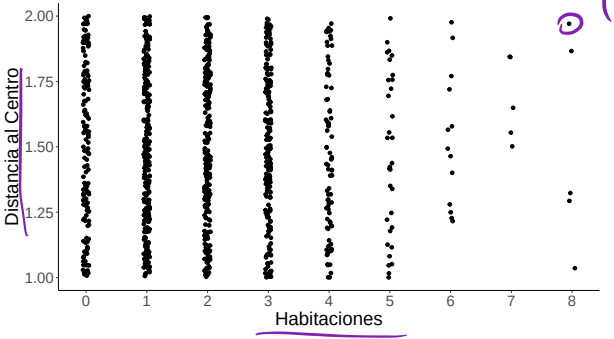
Motivación

- Queremos predecir:

x_i

$$Precio = f(habitaciones, Distancia\,al\,CBD) \tag{5}$$

(hab, DCBD, precio)



Motivación

- Podemos asumir f es lineal

$$f(\mathbf{x}_i;\beta) = \beta_0 + \beta_1 Habitaciones_i + \beta_2 DCBD_i + u_i \tag{6}$$

$$+ \beta_3 Hab^2 + \beta_4 Hab \times DCBD + \ldots$$

- o ajustar un modelo flexible e interpretable como son los árboles

$$f(x_i;\{R_j,\gamma_j\}_1^J) = T(x_i;\{R_j,\gamma_j\}_1^J) \tag{7}$$

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap

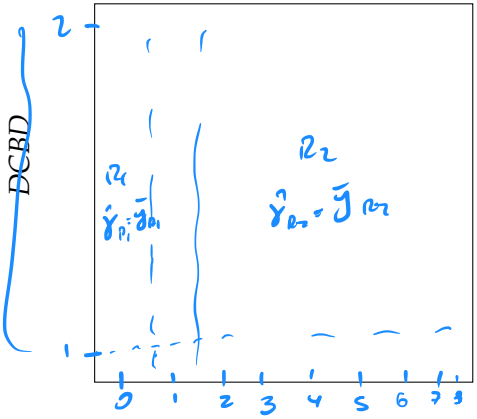
¿Qué hacen?

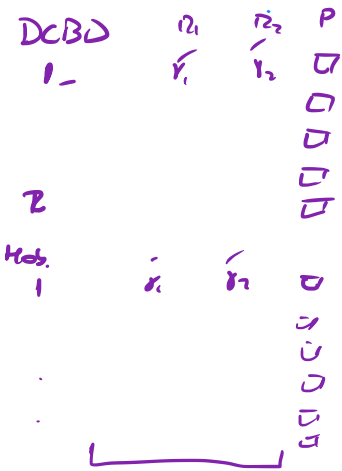
"Recursive binary splitting"

$$\min_{j,s}\left[\min_{\gamma_{R_1}} \sum (y-\gamma_{R_1})^2 + \min_{\gamma_{R_2}} \sum (y-\gamma_{R_2})^2\right]$$

Greedy

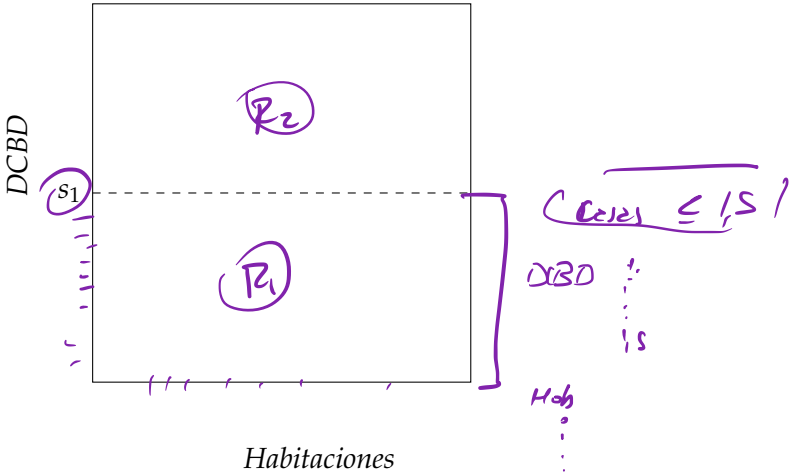
Voraz





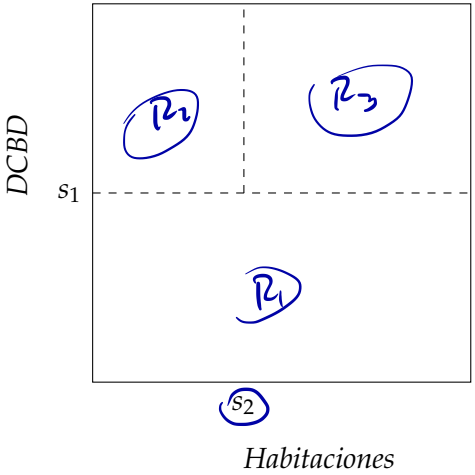
¿Qué hacen?

"Recursive binary splitting"

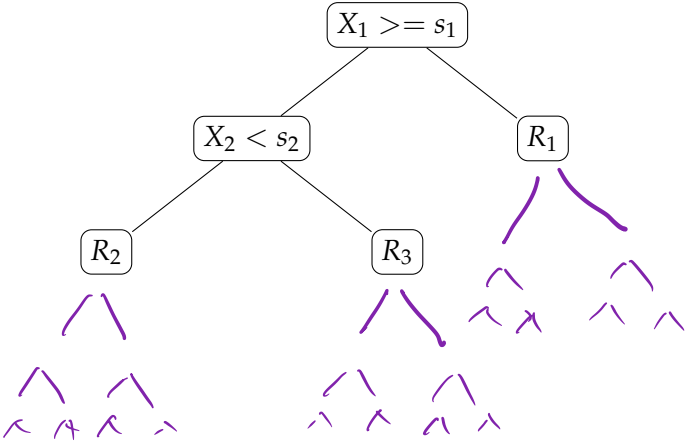


¿Qué hacen?

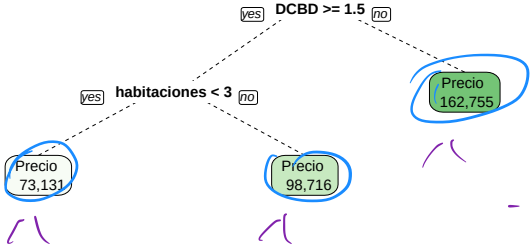
"Recursive binary splitting"

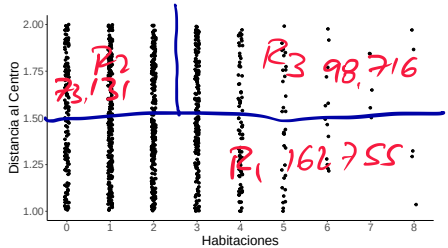


¿Qué hacen?



¿Qué hacen?


yes
DCBD >= 1.5
no
yes
habitaciones < 3
no
Precio
73,131
Precio
98,716
Precio
162,755


Distancia al Centro
2.00
1.75
1.50
1.25
1.00
0
1
2
3
4
5
6
7
8
Habitaciones

¿Cómo construimos un árbol?

- El problema de optimización

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{8}$$

- es ahora

$$\{\hat{R}_j, \hat{\gamma}_j\}_1^J = \underset{\{R_j, \gamma_j\}_1^J}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, T(\mathbf{x}_i; \{R_j, \gamma_j\}_1^J)) \right\} \tag{9}$$

¿Cómo construimos un árbol?

$X = (x_1, x_2)$

- Datos: $y_{n\times 1}$ y $X_{n\times k}$
- Definiciones
  - j es la variable que parte el espacio y s es el punto de partición
  - Defina los siguientes semiplanos

$$R_1(j,s) = \{X|X_j \leq s\} \ \& \ R_2(j,s) = \{X|X_j > s\} \tag{10}$$

  - El problema: usando una "perdida cuadrática" buscar la variable de partición X_j y el punto s de forma tal que:

$$\min_{j,s}\left[\min_{\gamma_{R_1}} \sum_{x_i\in R_1(j,s)} (y-\gamma_{R_1})^2 + \min_{\gamma_{R_2}} \sum_{x_i\in R_2(j,s)} (y-\gamma_{R_2})^2\right] \tag{11}$$

¿Cómo construimos un árbol?

- ¿Cuál es la solución?

$$\min_{\gamma_{R_1}} \sum_{\underline{x_i \in R_1(j,s)}} (y_i - \gamma_{R_1})^2$$

$$\sum_i (y_i - \gamma_{R_1})(-1) = 0$$

$$-\sum_i y_i + n_1 \gamma_{R_1} = 0$$

$$\hat{\gamma}_{R_1} = \sum_{i \in R_1} y_i \cdot \frac{1}{n_1}$$

¿Cómo construimos un árbol?

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap

Árboles: Problema

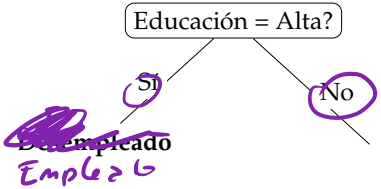
CART

- ¿Está desempleado?

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
Alta	28	No
Baja	30	Sí
Baja	35	No
Alta	40	No
Alta	38	No

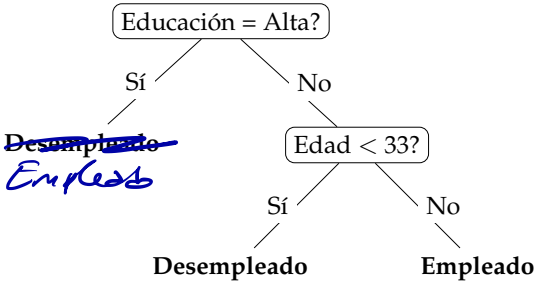
Árboles: Problema

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
Alta	28	No
Baja	30	Sí
Baja	35	No
Alta	40	No
Alta	38	No



Árboles: Problema

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
~~Alta~~	~~28~~	~~No~~
Baja	30	Sí
Baja	35	No
~~Alta~~	~~40~~	~~No~~
~~Alta~~	~~38~~	~~No~~



¿Cómo construimos un árbol de decisión?

- Regiones lo más "puras" posibles
  - **Regresión**: minima varianza
  - **Clasificación**: ?) varianza

$$\sum (y-\bar{y})^2$$

$$\begin{cases} 1 \\ 0 \end{cases} \quad p(1-p)$$

¿Cómo construimos un árbol de decisión?

Problemas de clasificación

Edad	Desempleado?
22	No
25	Sí
30	No
35	Sí
40	Sí

Edad Imp

22 ✓

25 ✓

30 ✓

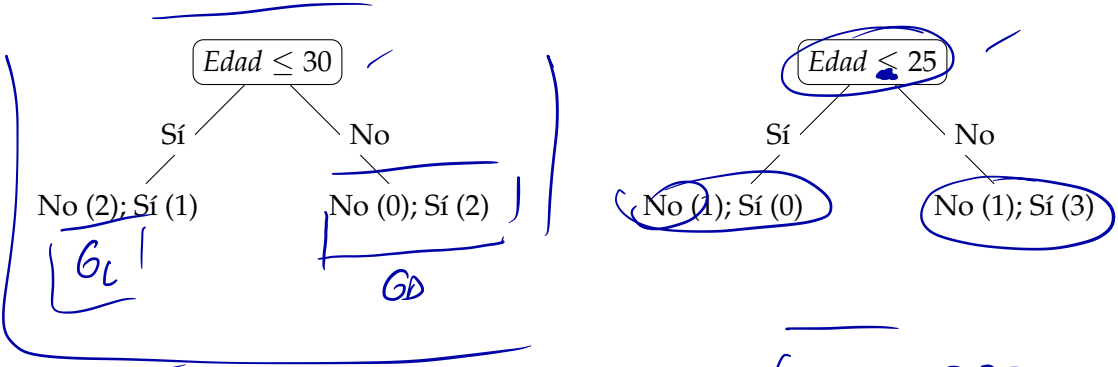
35 ✓

40 ✓

¿Cómo construimos un árbol de decisión?

Problemas de clasificación

- ¿Cuál de los dos cortes es mejor?



G_L

G_D

$$G_{Arbol} = \frac{1}{n_L} G_L + \frac{1}{n_R} G_R$$

$$G_{Arbol} = 0{,}30$$

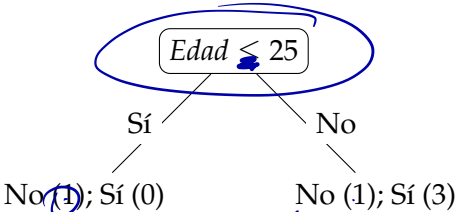
¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Medidas de Impureza

- Medidas de impureza dentro de cada hoja:
  - Índice de Gini : $G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$
  - Entropía : $H = -\sum_{k=1}^{K} \hat{p}_{mk} log(\hat{p}_{mk})$
- Se define la impureza de un árbol por el promedio ponderado de las impurezas de cada hoja. El ponderador es la fracción de observaciones en cada hoja.

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Impureza — Corte B: *Edad* ≤ 25



$\frac{1}{5}$

$p_1 \quad (1-p_1) \qquad p_2 \quad (1-p_2)$

$\frac{1}{4}\left(1-\frac{1}{4}\right) + \frac{3}{4}\left(1-\frac{3}{4}\right)$

$\left(\frac{1}{4}\cdot\frac{3}{4} + \frac{3}{4}\cdot\frac{1}{4}\right)$

Hoja izq. ($n_L = 1$): No(1), Sí(0) — *pura*

$$G_L = 0$$

Hoja der. ($n_R = 4$): No(1), Sí(3)

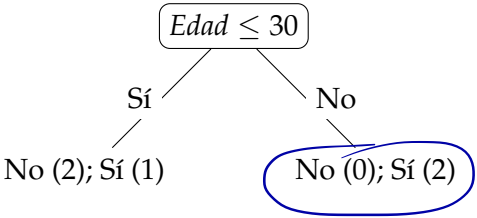
$$G_R = 2 \cdot \frac{1}{4} \cdot \frac{3}{4} = \frac{3}{8} = 0.375$$

Impureza ponderada:

$$G_B = \frac{1}{5} \cdot 0 + \frac{4}{5} \cdot \frac{3}{8} = \frac{3}{10} = \mathbf{0.30}$$

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Impureza — Corte A: $Edad \leq 30$



Hoja izq. ($n_L = 3$): No(2), Sí(1)

$$G_L = 2 \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{4}{9} \approx 0.44$$

Hoja der. ($n_R = 2$): No(0), Sí(2) — *pura*

$$G_R = 0$$

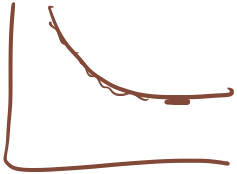
Impureza ponderada:

$$G_A = \frac{3}{5} \cdot \frac{4}{9} + \frac{2}{5} \cdot 0 = \frac{4}{15} \approx \mathbf{0.27}$$

¿Cómo construimos un árbol de decisión?

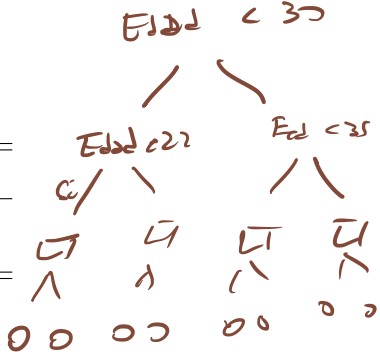
Problemas de clasificación. Comparación

- Ambas medidas eligen el mismo corte óptimo:



Corte	Gini	Entropía
Edad ≤ 30	**0.27**	**0.55**
Edad ≤ 25	0.30	0.65

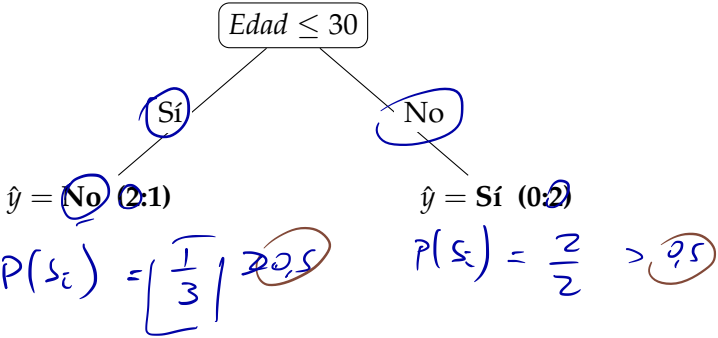
35 ☐
40 ☐



¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Predicción

- Predicción



Medidas de Impureza: Gini vs. Entropía

Índice de Gini

$$G = 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k)$$

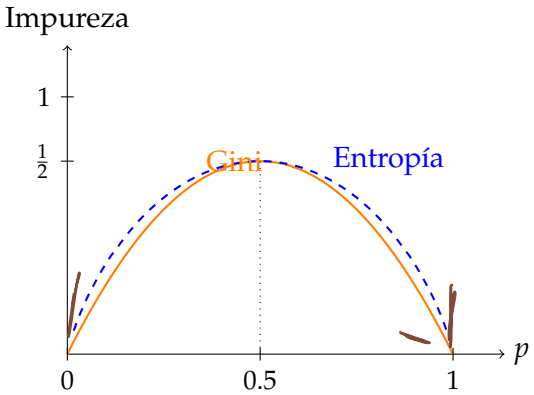
Entropía (cross-entropy)

$$H = -\sum_k p_k \log_2 p_k$$

Propiedades comunes

- **Mínimo** $= 0$ en nodos puros ($p_k = 1$)
- **Máximo** en distribución uniforme
- Ambas son **cóncavas**
- Ambas producen **resultados similares** en la práctica

Medidas de Impureza: Gini vs. Entropía



Caso binario ($K = 2$, prob. clase $1 = p$):

$$G(p) = 2p(1-p)$$
$$H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

Diferencia clave:

- Entropía **crece más rápido** cerca de 0 y 1
- Gini es más **plana en los extremos**

$\Rightarrow$ Entropía penaliza más la presencia de clases raras

Medidas de Impureza: Gini vs. Entropía

Gini vs. Entropía: ¿Cuál usar?

Gini (CART clásico)

- Más rápido (evita logaritmos)
- Menos sensible a clases raras
- Tiende a aislar la clase más frecuente en una rama
- **Predeterminado en R**

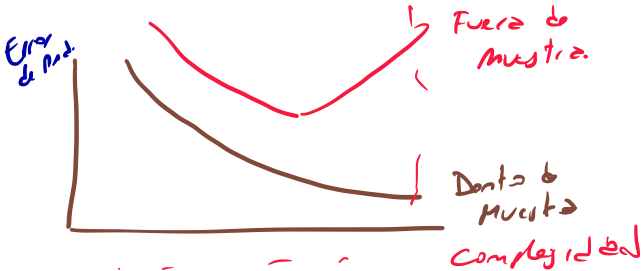
Entropía

- Más sensible a pureza extrema
- Penaliza más la presencia de clases raras
- Puede favorecer splits que **aíslan clases minoritarias**
- Útil cuando la clase rara importa

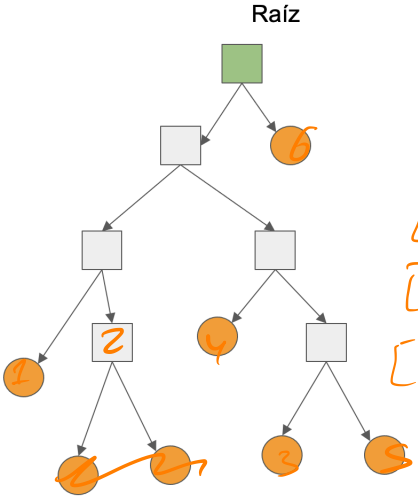
En la práctica, la diferencia en rendimiento suele ser pequeña. Con clases desbalanceadas, a menudo es más importante ajustar los pesos de las clases o el umbral de decisión que cambiar la métrica de impureza.

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap



Sobreajuste



ERROR Prod

[T] = 7

[T] = 6

[T] = 5

[ ] = 4

C

Sobreajuste. Algunas soluciones

- Fijar la profundidad del árbol.
- Fijar la mínima cantidad de datos que están contenidos dentro de cada hoja.
- Pruning (poda).
  - Dejar crecer un árbol muy grande T_0
  - Luego cortarlo obteniendo sub-árbol (*subtree*)
  - Como cortarlo?

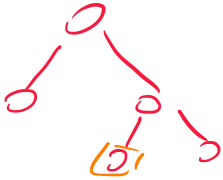
Pruning (poda)

- No es posible calcular el error de predicción usando cross-validation para cada sub-árbol posible
- Solución: *Cost complexity pruning (cortar las ramas mas débiles)*
  - Indexamos los arboles con T.
  - Un sub-árbol $T \in T_0$ es un árbol que se obtuvo colapsando los nodos terminales de otro árbol (cortando ramas).
  - $[T]$ = número de nodos terminales del árbol T

$[T]$

Pruning (poda)

$\alpha = 0{,}01$



- Cost complexity del árbol T

$$C_\alpha(T) = \sum_{m=1}^{|T|} n_m Q_m(T) + \alpha |T| \tag{12}$$

$C_{\alpha=0{,}01} = 1 + 0{,}001\,[5]$

$\alpha = 0{,}01$

$[4]$

- donde $Q_m(T) = \frac{1}{n_m} \sum_{x_i \in R_m} (y_i - \hat{y}_m)^2$ para los árboles de regresión
- $Q_m(T)$ penaliza la heterogeneidad dentro de la regresión y α el número de regiones
- Objetivo: para un dado α, encontrar el pruning óptimo que minimice $C_\alpha(T)$

$[T]$

$C_{\alpha=0{,}02}$

Pruning (poda)

- Mecanismo de búsqueda para T_α (pruning óptimo dado α).

  Resultado: para cada α hay un sub-árbol único T_α que minimiza $C\alpha$ (T).

  - Eliminar sucesivamente las ramas que producen un aumento mínimo en $\sum_{m=1}^{[T]} n_m Q_m(T)$
  - Se colapsa hasta el nodo inicial pero va a través de una sucesión de árboles
  - T_α pertenece a esta secuencia. (Breiman et al., 1984)

Pruning (poda)

Algoritmo Completo

1. Utilizamos particiones recursivas binarias para hacer crecer el árbol
2. Para un dado α, aplicamos *cost complexity pruning* al árbol para obtener la secuencia de los subarboles como α.
3. Utilizamos K-fold cross-validation para elegir α.
4. Tenemos entonces una secuencia de subarboles para distintos valores de α
5. Elegimos el α y el subárbol que tienen el menor error de predicción.

Pruning (poda) para Clasificación

- La misma idea aplica para árboles de clasificación: dejar crecer T_0 y luego podar.
- La medida de impureza $Q_m(T)$ se reemplaza por Gini o Entropía:

$$C_\alpha(T) = \sum_{m=1}^{[T]} n_m \, Q_m(T) + \alpha [T] \tag{13}$$

donde

$$Q_m(T) = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad \text{(Gini)}$$

- $n_m Q_m(T)$: impureza total de la hoja m.
- $\alpha\,[T]$: penalización por complejidad del árbol ($[T]$ = número de hojas).
- Un α grande favorece árboles simples; $\alpha = 0$ recupera T_0.

Ejemplo

photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1 Recap

2 Árboles
- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

3 Bagging y Random Forests
- Bagging
- Random Forests

4 Recap

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap

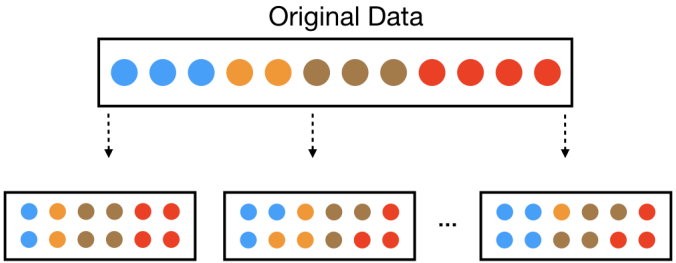
Bagging

- Problema con CART: pocos robustos.
- Podemos mejorar mucho el rendimiento mediante la agregación
- Idea: la varianza del promedio es menor que la de una sola predicción.

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).

Bagging



Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

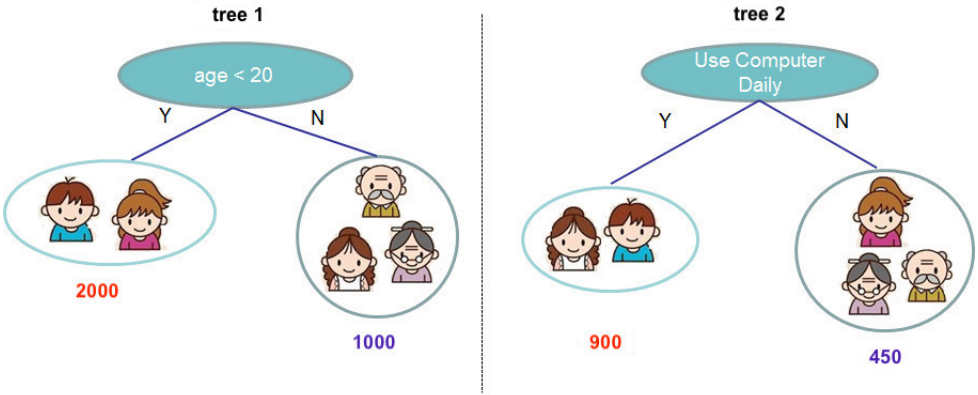
$$\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x) \tag{14}$$

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

$$\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x) \tag{14}$$

Bagging



f ( ) = (2000 + 900)/2 = 1450 f ( ) = (1000 + 450)/2 = 725

Agenda

1. Recap
2. Árboles
   - Árboles de Regresión
   - Árboles de Clasificación
   - Sobreajuste
3. Bagging y Random Forests
   - Bagging
   - Random Forests
4. Recap

Random Forests

- Problema con el bagging: si hay un predictor fuerte, diferentes árboles son muy similares entre sí.
- Bosques (forests): reduce la correlación entre los árboles en el boostrap.
- Si hay p predictores, en cada partición use solo $m < p$ predictores, elegidos al azar.
- Bagging es forests con $m = p$ (usando todo los predictores en cada partición).
- m es un hiper-parámetro

Agenda

1 Recap

2 Árboles
- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

3 Bagging y Random Forests
- Bagging
- Random Forests

4 Recap

Recap

Árboles (CART)

- Particiones recursivas binarias del espacio de X
- **Regresión**: minimiza RSS en cada corte
- **Clasificación**: minimiza Gini o Entropía
- Problema: alta varianza, sobreajuste

Pruning

- Crecer T_0 grande y podar
- Criterio:

$$C_\alpha(T) = \sum_m n_m Q_m + \alpha[T]$$

- **Weakest link**: colapsar el nodo mas debil
- Genera secuencia $T_0 \supset T_1 \supset \cdots$
- $\hat{\alpha}$ se elige por cross-validation

Bagging / RF

- **Bagging**: promediar árboles sobre muestras bootstrap reduce varianza
- **Random Forests**: en cada corte, usar subconjunto predictores aleatorios
- Decorrelaciona los árboles $\Rightarrow$ mejora bagging
- Precio: se pierde interpretabilidad